

Hàm

- Chúng ta không thể viết tất cả chương trình đặt trong hàm main được , mặc dù code vẫn chạy , nhưng sau này sẽ khó bảo trì. Hàm sinh ra để giải quyết bài toán này
- Bản chất hàm giúp chúng ta gộp những đoạn code lặp đi lặp lại đóng gói dưới một cái tên dễ nhớ : như bat_den(), tat_den() , xoay_trai() ... và ta chỉ việc gọi những hàm này khi cần
- Ngoài việc giúp chúng ta tối ưu, tái sử dụng code . Hàm là công cụ giúp chúng ta chia chương trình thành những chương trình con với mục đích giải quyết một vấn đề phức tạp . Phương pháp lập trình này gọi là top-down

Cú pháp cơ bản hàm

Hàm gồm có các thành phần cơ bản

- kiểu dữ liệu trả về: là kết quả sau khi hàm thực hiện tính toán trả về cho chúng ta. Có thể không có giá trị trả về Ví dụ:
 - hàm tính diện tích thì kết quả cần trả về là diện tích
 - hàm bật led thì không cần trả về vì ta chỉ thực thi mỗi tính năng bật led
 - hàm bật led ta cũng có thể thiết kế để nó trả về kết quả bật thành công hay không
- tên hàm: tên đại diện cho hàm, chúng ta muốn dùng hàm sẽ gọi hàm thông qua tên hàm
- tham số của hàm: những giá trị ta truyền vào trong hàm để thực hiện tính toán

Tùy nhu cầu bài toán chúng ta chọn hàm có giá trị trả về hoặc không có giá trị trả về

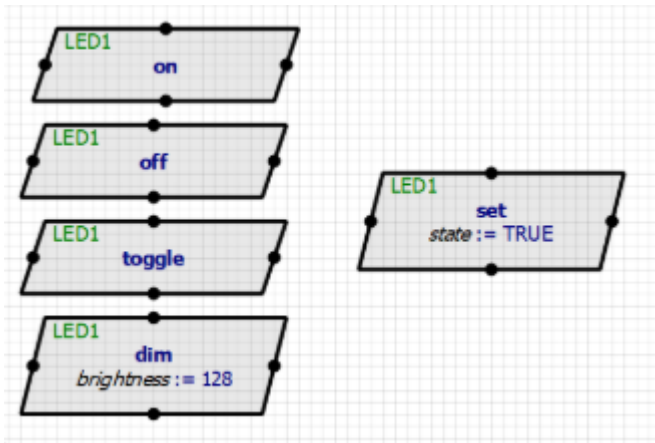
hàm có không có giá trị trả về

```
void tên_hàm (kiểu_dữ_liệu tham_số_1, kiểu_dữ_liệu tham_số_2)
{
    //lệnh thực thi bên trong
}
```

void từ khóa báo hiệu hàm không có giá trị trả về

Ví dụ

Xét cách triển khai này chính là các hàm hỗ trợ led chúng ta đã dùng trong protues



Khối lệnh	Khi nào nên dùng?
On / Off	Khi bạn chỉ cần điều khiển đơn giản, cố định (ví dụ: nhấn nút A thì bật LED).
Toggle	Dùng để đảo trạng thái (đang sáng thành tắt và ngược lại), cực kỳ hữu ích cho nút nhấn đơn.
Dim	Dùng khi bạn muốn điều chỉnh độ sáng (PWM) theo số lượng cụ thể (0-255).
Set	Dùng khi bạn muốn lập trình chuyên nghiệp hơn, truyền một giá trị logic từ một hàm khác hoặc một cảm biến vào trực tiếp cho LED.

Ta thấy đây thực chất người ta đã đóng gói thành các hàm tiện ích như bật (khối on) , tắt (off) , đảo trạng thái (khối toggle) , chỉnh độ sáng (khối dim) và thiết lập sáng tắt dựa vào điều kiện (khối set) . Với kiến thức hàm ta có thể triển khai lại như sau

- Hàm on với tham số pin

```
void on(int pin)
{
    //thực hiện lệnh xuất mức cao ra chân led: digitalWrite(pin,ON) sẽ học ở phần
    arduino)
    digitalWrite(pin,HIGH); //sẽ học ở phần arduino
}
```

- Hàm off với tham số pin

```
void off(int pin)
{
    //thực hiện lệnh xuất mức thấp ra chân led: digitalWrite(pin,LOW) sẽ học ở
```

```
phần arduino)
    digitalWrite(pin, LOW); //sẽ học ở phần arduino
}
```

- giả sử sau khi có hàm on(), off() ta xây dựng hàm toggle với tham số **pin**

```
void toggle(int pin)
{
    // thực hiện đọc trạng thái hiện tại led đang ở ON , OFF thông qua lệnh
    digitalWrite(pin) lưu vào biến trạng thái trang_thai
    int trang_thai= digitalWrite(pin); //học ở phần arduino

    if(trang_thai==0)
    {
        //hiện tại led đang tắt
        //gọi hàm on() để bật led
        on(pin);
    }
    else
    {
        off(pin);
    }
}
```

- Hàm dim nhận **tham số** từ 0-255 để xuất điện áp chạy từ 0-5V ra led.
- Ta xây dựng hàm dim với tham số **pin** , **độ sáng** chạy từ 0 cho đến 255

```
void dim (int pin, int do_sang)
{
    //thực hiện lệnh xuất tín hiệu điện áp thay đổi ra chân led: sử dụng
    analogWrite(pin, do_sang) ---học ở phần arduino
    digitalWrite(pin, do_sang); // học ở arduino
}
```

- Hàm set nhận giá trị đầu vào là kết quả của biểu thức điều kiện và sẽ sáng led khi kết quả là true và tắt led khi kết quả là false. Ví dụ nhiệt độ >50 là một biểu thức điều kiện nó trả ra là true/ false

Ta xây dựng hàm dim với tham số đầu vào là **pin** , **ket_qua_dieu_kien**

```
void set(int pin , int ket_qua_dieu_kien)
{
    if (ket_qua_dieu_kien==1)
    {
        on(pin);
    }
    else
```

```

    {
        off(pin);
    }
}

```

ham có giá trị trả về

```

kiểu_dữ_liệu tên_hàm (kiểu_dữ_liệu tham_số_1, kiểu_dữ_liệu tham_số_2)
{
    //lệnh thực thi bên trong
    //sau khi thực thi xong lưu vào biến ket_qua
    return ket_qua;
}

```

- Bắt buộc phải có kiểu dữ liệu trả về
- cuối lệnh bắt buộc phải có lệnh return để trả về kết quả

Ví dụ 1

- xây dựng hàm tính diện tích

```

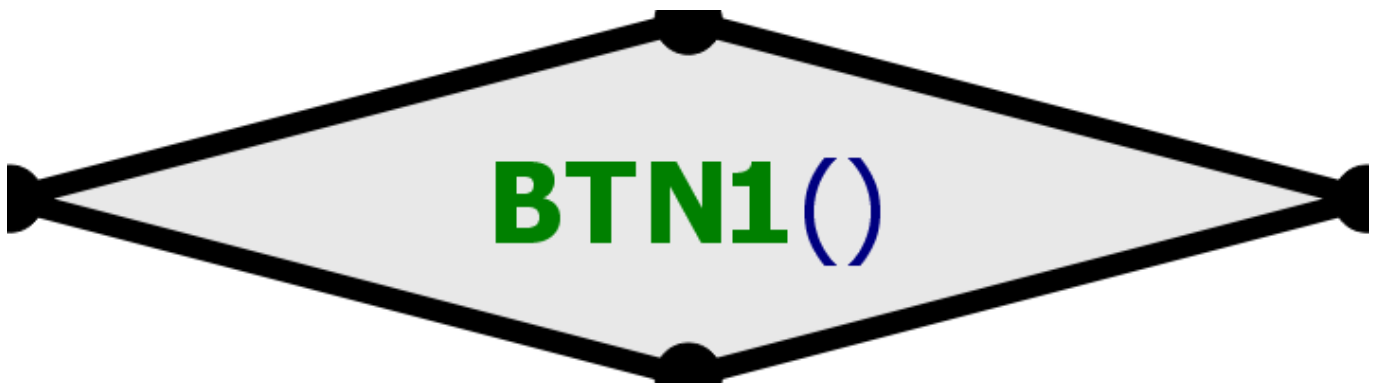
double tinh_dien_tich(double chieu_dai, double chieu_rong)
{
    double ket_qua = chieu_dai*chieu_rong;
    return ket_qua;
}

```

nhân xét sau khi thực hiện tính toán trả về biến kết quả . biến kết quả kiểu phải là kiểu double để khớp với kiểu trả về của hàm là double

Ví dụ 2

- Khảo sát khối button chúng ta đã từng làm trong protues



Ta thấy khối này sẽ thực hiện kiểm tra button có được nhấn hay không . Bên trong nó là hàm BTN1() đây là hàm để đọc trạng thái chân button và trả ra kết quả true/false. Sau khi có kết quả kết hợp với khối điều kiện

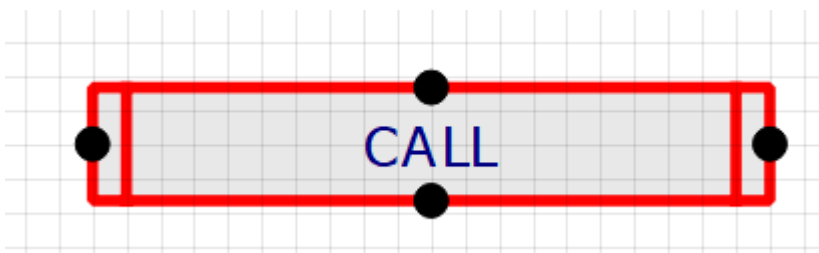
- Xây dựng khối đọc kết quả nút nhấn

```
int doc_nut_nhan(int pin)
{
    //thực hiện đọc trạng thái nút nhấn ON, OFF. thông qua lệnh digitalWrite(pin)
    và lưu vào biến trạng thái -- sẽ học phần arduino
    int trang_thai = digitalWrite(pin)
    return trang_thai;
}
```

- cải tiến thêm tính hằng chống dội vào hàm đọc nút nhấn

```
int doc_nut_nhan_va_chong_doi(int pin)
{
    int trang_thai = digitalWrite(pin);
    //kiểm tra nút có nhấn chưa , nếu nhấn thì chúng ta chống dội ở đây luôn
    if (trang_thai==1)
    {
        //nút được nhấn
        //chống dội : dùng vòng lặp while để chờ sự kiện thả tay
        while(1)
        {
            int trang_thai_2 = digitalWrite(pin);
            if (trang_thai_2==0)
            {
                //đã thả tay - hoàn thành quá trình nhấn nút
                return 1;
            }
        }
    }
    return 0;
}
```

Truyền tham số vào hàm (lời gọi hàm)



- Sau khi định nghĩa hàm xong , ta cần gọi hàm để sử dụng.
- Gọi hàm cần chú ý các tham số truyền vào đúng kiểu và đúng thứ tự

Ví dụ 1:

- Muốn bật đèn đang đầu vào chân 1 : gọi hàm on(1)
- Muốn bật đèn đang đầu vào chân 3 : gọi hàm on(3)
- Muốn đọc nút nhấn đang đầu vào chân 5 : gọi hàm doc_nut_nhan_va_chong_doi(5)
- Muốn tính diện tích : chiều dài =10, chiều rộng =7.5 : gọi hàm tinh_dien_tich(10,7.5)

****chú ý nếu ta gọi hàm on (1.5) tức là truyền 1.5 vào biến pin trong hàm , mà biến pin đang kiểu nguyên không phù hợp để nhận giá trị 1.5 nên chương trình bị lỗi ngay ****

Ví dụ 4

- Kết hợp tất cả lại ta có bài toán sau: Giả sử **nút nhấn 1** đầu vào chân **3** , **đèn** đầu vào chân **4**. ta muốn viết chương trình bật tắt đèn mỗi lần nhấn nút ta làm như sau

```
if(doc_nut_nhan_va_chong_doi(3)==1)
{
    toggle(4); //gọi hàm thông qua tên hàm
}
```

ở đây gọi ra hai hàm

- đầu tiên gọi ra hàm doc_nut_nhan_va_chong_doi(3) để lấy kết quả trả về xem có nhấn nút hay chưa, kết quả này đem so sánh với 1
- toggle(4) : đảo led đang gắn chương 4

Chương trình rất sạch và gọn gàng khi thiết kế hàm hợp lý